MCF Deep Learning, Hilary 2026
Problem Sheet 2: Convolutional Neural Networks.

1. Derive the stochastic gradient descent algorithm for a single-layer fully-connected neural network with dropout probability p and learning rates η_n, used for binary classification.

$$
\begin{aligned}
Z^1 &= W^1 x + b^1, \\
A_i &= R_i \sigma\left(Z_i^1\right), \\
Z^2 &= W^2 A + b^2, \\
[f(x,\theta)]_m = \mathbb{P}(Y = m) &= \frac{e^{Z_m^2}}{\sum_{m'=0}^{1} e^{Z_{m'}^2}},
\end{aligned}
$$

where $x \in \mathbb{R}^d, W^1 \in \mathbb{R}^{L\times d}, b^1 \in \mathbb{R}^L, A \in \mathbb{R}^L, W^2 \in \mathbb{R}^{2\times L}, b^2 \in \mathbb{R}^2, \sigma : \mathbb{R} \to \mathbb{R}, Z^2 \in \mathbb{R}^2$, and $Y \in \{0,1\}$. The loss function is the cross-entropy error (i.e., the negative log-likelihood). The parameters are $\theta = \{W^1, W^2, b^1, b^2\}$ .R_i is a Bernoulli random variable.

2. We would like to classify the digits from the MNIST dataset using a neural network. The network contains a single convolution layer followed by a fully connected layer for output. You should complete the homework in the Jupyter notebook `Homework_#2_Q2.ipynb`, and submit the completed notebook for grading.

   (a) i. Implement the `Convolution` function, which should take the following as input:
   - `X`, the image, as a 2D array, and
   - `F`, as a 3D array that contains `F.shape[0]` filters, each with size `(F.shape[1], F.shape[2])`.

   The output should be an 3D array of shape `(F.shape[0], dim_x, dim_y)` that represents convolved images of shape `(dim_x, dim_y)` from `F.shape[0]` channels.

   ii. We shall now test the Convolution operator. Your task is to figure out what the filters below do to the testing image. The testing image is `radcam.jpg`, which you should import into the Jupyter notebook.
   The filters are included by the following 3D array:

```
test_F = np.array([
    [[1/16, 1/8, 1/16], [1/8, 1/4, 1/8], [1/16, 1/8, 1/16]],
    [[0, -1, 0], [-1, 4, -1], [0, -1, 0]],
    [[0, -1, 0], [-1, 5, -1], [0, -1, 0]],
    [[1, 2, 1], [0, 0, 0], [-1, -2, -1]],
])
```

   (b) Following the instructions from the Jupyter notebook to implement and train the convolution network **from scratch** in Python for the MNIST dataset (i.e. no PyTorch). The neural network should be trained on the Training Set using stochastic gradient descent or mini-batch stochastic gradient descent.

3. We would like to reproduce some results from a paper [1] that investigate how dropout affect the ability of a convolutional neural network (CNN) to classify pictures from the CIFAR10 dataset. You should

   - either complete this question in the Jupyter notebook `Homework_#2_Q3.ipynb`, and submit the completed notebook for grading,
   - or use the Jupyter notebook to create Python files and run them in the University ARC cluster, and submit *all* codes for grading. There will be further guidance in the question to guide you through the process, but feel free to follow the ARC cluster guide as well.

   As the CNN is large, you should run your code with a GPU. You can request GPU in both Google Colab or University ARC cluster.

   The precise architecture is provided by Table 1 on the next page.

   (a) Fill in the output sizes in the Table 1 on the next page.

   (b) The example code from `Homework_#2_Q2.ipynb` is provided for this question, which starts with the following data augmentation steps:

   ```
   transformations = transforms.Compose([
                     transforms.RandomHorizontalFlip(p=0.5),
                     transforms.RandomVerticalFlip(p=0.5),
                     transforms.ToTensor(),
                     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
   ```

   Explain what augmentations are done, and what are the benefits of performing such augmentations?

   (c) Complete the example code from `Homework_#2_Q2.ipynb` to implement neural networks of the same architecture, one with dropout of rates as specified in the table and one without. (**Important**: the dropout rates here refers to the probability of node *being kept*, but Pytorch define the rates as the probability of node *being set to zero*. Be careful of this when you implement the CNN with dropout.)

   (d) Train those neural networks (with/without dropout) and compare their test accuracies. You should reserve 5000 images for validation. For each of the models, report the validation accuracies across training and the final test accuracy.

   Hint:

   - the authors include momentum 0.95 applied to each layers, so the parameter updates $\Delta\theta_k$ evolves as

   $$\Delta\theta_k = 0.95\Delta\theta_{k-1} + (1 - 0.95)\nabla\ell_k(\theta_{k-1}),$$

   where $\ell_k(\cdot)$ is the loss evaluated at batch k. Note how to set up this in Pytorch's implementation of SGD.

   - the author also include max-norm regularisation (gradient clipping) with parameter's ℓ^2-norm ≤ 4, which could be done by running this before `optimiser.step()`:

[1] N. Srivastava et. al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. In Journal of Machine Learning Research 15 (2014) 1929-1958.

```
nn.utils.clip_grad_norm_(model.parameters(), max_norm)
```

- it is a good practice to save the model parameters after training using the `torch.save()` method, and load the parameters using `torch.load()` method for post-training analysis, i.e., parts (e) and (f). Check documentation on how this could be precisely done.

(e) Plot, using the example code from the Jupyter notebook, some images from the dataset with their predicted and true labels. Comment on which kind(s) of images are often misclassified.

(f) The test accuracies could be computed by:

i. either the heuristic method,

ii. or the Monte-Carlo sampling.

Use the example code from the Jupyter notebook, compare the test accuracies obtained from (i) and (ii) with various sample sizes.

Layer	Output shape
(dropout $p = 0.9$)	(3,32,32)
convolution (96 output channels, 5x5, stride=1, padding=2), ReLU	$\downarrow$
max-pooling (3x3 region, stride=2, padding=0)	$\downarrow$
(dropout $p = 0.75$)	
convolution (128 output channels, 5x5, stride=1, padding=2), ReLU	$\downarrow$
max-pooling (3x3 region, stride=2, padding=0)	$\downarrow$
(dropout $p = 0.75$)	
convolution (256 output channels, 5x5, stride=1, padding=2), ReLU	$\downarrow$
max-pooling (3x3 region, stride=2, padding=0)	$\downarrow$
(dropout $p = 0.5$)	
flattening to 1D array	$\downarrow$
fully connected, ReLU	$\downarrow$
(dropout $p = 0.5$)	2048
fully connected, ReLU	$\downarrow$
(dropout $p = 0.5$)	2048
fully connected, Softmax	$\downarrow$
	10

Table 1: Architecture of the desired CNN

4. The following are conceptual questions which can be answered with a short paragraph.

   (a) i. Recall the RMSProp algorithm:

$$r^{(k+1)} = \rho r^{(k)} + (1-\rho) g^{(k)} \odot g^{(k)},$$
$$\theta^{k+1} = \theta^{(k)} - \frac{\epsilon}{\delta + \sqrt{r^{(k)}}} g^{(k)}.$$

   What are the effects of the different hyperparameters in the RMSProp algorithm?

   ii. The AdaGrad [2] algorithm is another optimization method which satisfies:

$$r^{(k+1)} = r^{(k)} + g^{(k)} \odot g^{(k)},$$
$$\Delta\theta^{(k)} = -\frac{\epsilon}{\delta + \sqrt{r^{(k)}}} g^{(k)}$$
$$\theta^{k+1} = \theta^{(k)} + \Delta\theta^{(k)},$$

   What is the advantage of RMSProp over the AdaGrad algorithm?

   (b) We separately train M neural network models $f\left(x;\theta^1\right), \ldots f\left(x;\theta^M\right)$ using stochastic gradient descent. Furthermore, each time we train a new model the initial parameter is randomly initialized. Therefore, the trained models $\theta^1, \ldots, \theta^M$ are i.i.d. RVs. Is it mathematically correct to use the following ensemble model?

$$g(x) = f\left(x; \frac{1}{M}\sum_{m=1}^{M} \theta^m\right)$$

   Why or why not?

   (c) Suppose we randomly initialize a neural network $f(x;\theta)$ and train it for a long time with the final parameter estimate being θ^1. Suppose we again randomly initialize the same neural network model $f(x;\theta)$ and train it again for a long time with the final parameter estimate being θ^2. Will θ^1 and θ^2 always be similar?

   (d) Let $Y = f(X)$. The universal approximation theorem states that, for every $\epsilon > 0$, there exists a neural network $f(x;\theta)$ and a parameter choice θ^* such that

$$\mathbb{E}_{X,Y}\left[\|Y - f\left(X;\theta^*\right)\|\right] < \epsilon.$$

   If we estimate θ using stochastic gradient descent, will it converge to the θ^* satisfying (0.3)?

5. What would the formula be for a 3D convolution with stride, padding, and multiple output channels? The input is a 3D image with C_{in} input channels (i.e., $X \in \mathbb{R}^{d_x \times d_y \times d_z \times C_{\text{in}}}$).

[2] In fact the AdaGrad algorithm is not quite correct - the denominator should have $\sqrt{\delta + r^{(k)}}$ instead.